

正規表現を用いたパスワード強度チェッカーを作ってみよう

本日のゴール

正規表現が書けるようになる！

以下の正規表現が読めるようになるはず？！

```
/^(?!.*(.)\1\1)(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*(?:<=\d)[!@#$%^&*]).{8,}/;
```

もくじ

1. 導入 - 実装の流れ (5分)
2. 正規表現の基礎 - 基本構文を学ぶ (15分)
3. レベル1-2実装 - 長さ と 英小文字 (15分)
4. レベル3-4実装 - 連続文字チェックと複数条件 (15分)
5. レベル5実装 - 後読みで完成！ (15分)
6. まとめ (10分)

1. 導入

実装の流れ

パスワードチェッカー

- レベル1: 8文字以上
- レベル2: 英小文字を含む
- レベル3: 連続する同じ文字が3つ以上ない
- レベル4: 英大文字と数字を両方含む
- レベル5: 数字の直後に記号がある箇所を含む

順番にチェック強度を上げていく

対応する正規表現

- レベル1-2: 量指定子と先読み
- レベル3: キャプチャグループ、後方参照、否定先読み、アンカー
- レベル4: 先読みの組み合わせ
- レベル5: 後読み

正規表現を順番に学んでいきましょう！

注意

正規表現を学ぶための題材として、パスワードチェッカーを扱っています。
実際のパスワードバリデーションには、セキュリティの要件やユーザビリティを考慮する必要があります。

2. 正規表現の基礎

基本構文を学ぶ

- メタ文字
- 文字クラス
- 先読み

正規表現とは？

Regular Expression (Regex) - 文字列のパターンを表現する記法

使用例

-  メールアドレスのバリデーション
-  電話番号のフォーマット確認
-  ログファイルからのデータ抽出
-  パスワードの強度チェック

JavaScriptでの書き方

/ で囲む、もしくは `RegExp` コンストラクタを使用

```
const patternA = /正規表現パターン/;  
const patternB = new RegExp("正規表現パターン");  
  
patternA.test("検証したい文字列"); // true or false
```

```
// 例: 3文字以上の英小文字にマッチ  
const regex = /^[a-z]{3,}$/;  
console.log(regex.test("abc")); // true
```

今回はスラッシュ表記 `/` を使います。

3. レベル1-2実装

長さチェック & 英小文字

レベル1: 基本的な「メタ文字」

メタ文字	意味	例
<code>.</code>	任意の1文字	<code>/a.c/</code> → "abc", "a9c"
<code>*</code>	～が0回以上	<code>/ab*/</code> → "a", "ab", "abb"
<code>+</code>	～が1回以上	<code>/ab+/</code> → "ab", "abb"
<code>?</code>	～が0回か1回	<code>/ab?/</code> → "a", "ab"
<code>{n}</code>	ちょうどn回	<code>/a{3}/</code> → "aaa"
<code>{n,}</code>	n以上	<code>/a{2,}/</code> → "aa", "aaa"
<code>{n,m}</code>	n以上かつm以下	<code>/a{2,4}/</code> → "aa", "aaa", "aaaa"

※JavaScriptでは `{,n}` が使えません

パスワードチェックアプリを起動しよう

1. `deno run --allow-read --allow-net server.deno.js`
2. `localhost:8080` にアクセス
3. パスワード強度チェッカーページが表示されたら成功！

文字を入力すると、正規表現のチェック結果が✖/✔で表示されます。

メタ文字を使ってみよう

`project/password-checker.js` を開く

`checkLevel1` に正規表現を書いてみよう！

```
export function checkLevel1(password) {  
  return /a{4}/.test(password); // 例えば、aが4回連続するかチェック  
}
```

色々書いて試してみよう！

レベル1: 「文字クラス」

角括弧 `[]` で文字の集合を指定

パターン	意味	例
<code>[abc]</code>	a, b, c のいずれか	<code>/[abc]/</code> → "a", "b", "c"
<code>[a-z]</code>	英小文字	<code>/[a-z]/</code> → "a", "m", "z"
<code>[A-Z]</code>	英大文字	<code>/[A-Z]/</code> → "A", "M", "Z"
<code>[0-9]</code>	数字	<code>/[0-9]/</code> → "0", "5", "9"
<code>[a-zA-Z]</code>	英字 (大文字小文字)	<code>/[a-zA-Z]/</code> → "a", "Z"

文字クラスを使ってみよう

checkLevel1 に正規表現を書いてみよう！

```
export function checkLevel1(password) {  
  return /[a-zA-Z0-9]{8,}/.test(password); // 例えば、英数字が8文字以上かチェック  
}
```

色々書いて試してみよう！

レベル1: 8文字以上かチェック

条件

- パスワードが8文字以上であること

レベル1: 実装してみよう

ファイル: `project/password-checker.js`

```
export function checkLevel1(password) {  
  return /^(?!)/.test(password);  
}
```

- パスワードが8文字以上であること

レベル1: テストしてみよう

ブラウザで確認

1. `deno run --allow-read --allow-net server.deno.js`
2. `localhost:8080` にアクセス
3. パスワード入力欄に文字を入力

Deno でテスト

```
deno test project/password-checker.test.js -- --level=1
```

レベル1: 答え合わせ

```
export function checkLevel1(password) {  
  return /.{8,}/.test(password);  
}
```

- `.` : 任意の1文字
- `{8,}` : 8回以上の繰り返し

レベル2: 「先読み」

文字列が特定のパターンを含むかチェック

肯定先読み (=パターン)</p

文字列が指定のパターンを含むことを確認

```
/(?=[a-z]+)/.test("Pass") // true (英小文字を含む)  
/(?=[a-z]+)/.test("PASS") // false (英小文字を含まない)
```

肯定先読みの「肯定」: `===` のイメージ。反対に「否定」は `!==` のイメージ。

先読みの特徴

マッチ位置が進まない

例えば、小文字を含む && 4文字以上のパターンが書ける。

```
/(?=.*[a-z]).{4}/.test("Pass"); // true
```

複数の条件を組み合わせられる

```
/(?=.*[a])(?=.*[b])/.test("ab"); // true  
/(?=.*[a])(?=.*[b])/.test("ba"); // true  
  
/(?=.*[a])(?=.*[B])/.test("ab"); // false
```

レベル2: 英小文字を含む

条件

- 8文字以上
- 英小文字を1文字以上含む

レベル2: 実装してみよう

```
export function checkLevel2(password) {  
  return /(?!)/.test(password); // TODO  
}
```

レベル2: テストしてみよう

ブラウザで確認

1. `deno run --allow-read --allow-net server.deno.js`
2. `localhost:8080` にアクセス
3. パスワード入力欄に文字を入力

Deno でテスト

```
deno test project/password-checker.test.js -- --level=2
```

レベル2: 答え合わせ

```
export function checkLevel2(password) {  
  return /^(?=.*[a-z]).{8,}/.test(password);  
}
```

- `(?=.*[a-z])`
 - i. `.*` で任意の文字を0文字以上スキップ
 - ii. `[a-z]` 英小文字が1つ以上あればマッチ
 - iii. 位置は最初に戻る（先読みなので）
- `.{8,}`
 - 8文字以上をチェック

レベル1-2: 「マッチと繰り返し」

正規表現は「マッチ（文字クラス）」と「繰り返し（数量子）」の組み合わせ

- 「文字クラス」は、その文字に**マッチするか指定**
- 「数量子」は、マッチした文字が**何回繰り返されるか指定**

例:

`.{8,}` は、`.` が文字クラス、`{8,}` が数量子

`*[a-z]` は、`.` が文字クラス、`*` が数量子、`[a-z]` が文字クラスだけ

※ 文字クラスだけ、数量子なしはOK。数量子だけはNG

4. レベル3-4実装

連続文字チェックと複数条件

レベル3: 連続する同じ文字が3つ以上ない

条件

- 8文字以上
- 英小文字を含む
- 連続する同じ文字が3つ以上ない

例

- Paaassword1  (aが3回連続)
- Password11  (2回までOK)

レベル3: 「キャプチャグループ」「後方参照」

キャプチャグループ (パターン)

後方参照 \n

`(.)\1(.)\2` // 例: 同じ文字が2回連続する

- `(.)` : 任意の1文字をキャプチャ
- `\1` : キャプチャした文字への後方参照
 - 後方参照はキャプチャグループの番号を指定
- `(.)\1` = 同じ文字2回、`(.)\1\1` = 同じ文字3回

レベル3: 「否定先読み」

否定先読み (?!パターン)

文字列が指定のパターンを**含まない**ことを確認

```
/(?!.*[a-z])/ .test("Pass") // false (英小文字を含む)  
/(?!.*[a-z])/ .test("PASS") // true (英小文字を含まない)
```

肯定先読み「(?=パターン)」の反対。

レベル3: 「アンカー」

アンカー: 文字列の位置を指定する特殊な記号

^	// 文字列の開始
\$	// 文字列の終了

アンカーの役割

- `^` : 文字列の**最初**の位置を表す
- `$` : 文字列の**最後**の位置を表す
- 実際の文字にはマッチせず、位置だけを指定する

レベル3: 実装してみよう

```
export function checkLevel3(password) {  
  return /(?!)/.test(password); // TODO  
}
```

- 連続する同じ文字が3つ以上ない

レベル3: テストしてみよう

ブラウザで確認

1. `deno run --allow-read --allow-net server.deno.js`
2. `localhost:8080` にアクセス
3. パスワード入力欄に文字を入力

Deno でテスト

```
deno test project/password-checker.test.js -- --level=3
```

レベル3: 答え合わせ

```
export function checkLevel3(password) {  
  return /^(?!.*(.)\1\1)(?=.*[a-z]).{8,}/.test(password);  
}
```

- `(?!.*(.)\1\1)` : 連続3文字がないことを確認 (否定先読み)
- `(?=.*[a-z])` : 英小文字を含む (肯定先読み)
- `.{8,}` : 8文字以上
- `^` : 文字列全体をマッチ

レベル3: 答え合わせの解説

アンカー `^` がない場合、「連続3文字を含まない英字8文字以上」の文字列にもマッチしてしまう。

例えば、`Paaaasword11` は `aasword11` が条件を満たすためマッチしてしまう。

`^(?!.*(.)\1\1)` で文字列全体に連続3文字がないことを確認

レベル4: 英大文字と数字を両方含む

条件

- 8文字以上
- 英小文字を含む
- 連続する同じ文字が3つ以上ない
- **英大文字を含む**
- **数字を含む**

レベル4: 他の「文字クラス」

文字クラスにはショートハンドがあり、短く書けます。

- `\d` : 数字 = `[0-9]`
- `\w` : 英数字とアンダースコア = `[A-Za-z0-9_]`
- `\s` : 空白を表す文字 = `[\f\n\r\t\v]` と、Unicodeの空白文字

他にもありますが省略。詳しくは末尾の参考資料 MDN 文字クラスへ

レベル4: 実装してみよう

```
export function checkLevel4(password) {  
  return /(?!)/.test(password); // TODO  
}
```

- レベル3の条件に加えて
- 英大文字を含む
- 数字を含む

レベル4: テストしてみよう

ブラウザで確認

1. `deno run --allow-read --allow-net server.deno.js`
2. `localhost:8080` にアクセス
3. パスワード入力欄に文字を入力

Deno でテスト

```
deno test project/password-checker.test.js -- --level=4
```

レベル4: 答え合わせ

```
export function checkLevel4(password) {  
  return /^(!.*(.)\1\1)(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8,}/.test(password);  
}
```

- `(?=.*[A-Z])` : 英大文字を含む (肯定先読み)
- `(?=.*\d)` : 数字を含む (肯定先読み)
- `^(!.*(.)\1\1)(?=.*[a-z]){8,}` : 他はレベル3と同様

5. レベル5実装

後読みと位置指定で完成

レベル5: 数字の直後に記号がある箇所を含む

条件

- 8文字以上
- 英小文字を含む
- 連続する同じ文字が3つ以上ない
- 英大文字を含む
- 数字を含む
- **数字の直後に記号(!@#\$%^&*)がある箇所を含む**

レベル5: 「肯定後読み」

肯定後読み (?<=パターン)

```
/(?<=\d)[$]/.test("5$"); // true (直前が数字)
```

- (?<=\d) : 直前が数字であることを確認 (後読み)
- [\$] : 記号 \$ にマッチ

レベル5: 実装してみよう

```
export function checkLevel5(password) {  
  return /(?!)/.test(password); // TODO
```

- レベル4の条件に加えて
- 数字の直後に記号(!@#\$%^&*)がある箇所を含む

レベル5: テストしてみよう

ブラウザで確認

1. `deno run --allow-read --allow-net server.deno.js`
2. `localhost:8080` にアクセス
3. パスワード入力欄に文字を入力

Deno でテスト

```
deno test project/password-checker.test.js -- --level=5
```

レベル5: 答え合わせ

```
export function checkLevel5(password) {  
  return /^(?!.*(\.)\1\1)(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*(?:\d)[!@#$%^&*]).{8,}/  
    .test(password);  
}
```

- `(?=.*(?:\d)[!@#$%^&*])` : 数字の直後に記号がある箇所を含む
 - `(?=.*...)` : 先読みで存在確認
 - `(?<=\d)[!@#$%^&*]` : 直前が数字の記号
- 他はレベル4と同様

完成したチェッカー

収まらないので正規表現だけ抜粋

```
/{8,}/ // レベル1
```

```
/(?=.*[a-z]).{8,}/ // レベル2
```

```
/^(?!.*(.)\1\1)(?=.*[a-z]).{8,}$/ // レベル3
```

```
/^(?!.*(.)\1\1)(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}/ // レベル4
```

```
/^(?!.*(.)\1\1)(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*(?<=\d)[!@#$%^&*]).{8,}/ // レベル5
```

6. まとめ

今日学んだこと

正規表現の構文

-  メタ文字と量指定子 `.`, `*`, `+`, `?`, `{n,m}`、`\d`
-  文字クラス `[a-z]`, `[0-9]`
-  アンカー `^` と `$`
-  先読み `(?=...)` と否定先読み `(?!...)`
-  後読み `(?<=...)`
-  キャプチャグループ `()` と後方参照 `\1`

応用例: パスワード以外にも、色々使える

-  メールアドレスのバリデーション
 - `/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/`
 - (英数字か記号1文字以上)@(英数字か.-1文字以上).(英字2文字以上)
-  日本の携帯電話番号
 - `[0-9]{3}[-]?[0-9]{4}[-]?[0-9]{4}`
 - 3桁-4桁-4桁 (ハイフンはありませんし対応)
-  ログファイルからのデータ抽出
 - `/ERROR:\s(.*)/`
 - "ERROR: "の後の任意の文字列

さらに学ぶには

- [MDN Web Docs - 正規表現](#)
- [MDN Web Docs - 文字クラス](#)
- [正規表現視覚化ツール](#)
- [Qiita 正規表現入門 - 入門記事](#)
- [とほほの正規表現入門](#)